



Unión Europea

Fondo Social Europeo

El FSE invierte en tu futuro

PHP

OOP, Object Oriented Programming en PHP

Pepe

INSTITUT



**GENERALITAT
VALENCIANA**

Conselleria d'Educació,
Investigació, Cultura i Esport



GOBIERNO
DE ESPAÑA

MINISTERIO
DE EDUCACIÓN, CULTURA
Y DEPORTE

Continguts

1	Pròleg	3
2	Clases y Objetos	3

1 Pròleg

La **Programación Orientada a Objetos** (OOP, Object Oriented Programming) es un estilo de organizar el código que permite a los desarrolladores agrupar tareas similares en clases. Esto ayuda a que el código sea más fácil de mantener y a no repetirse (**DRY**, Don't Repeat Yourself).

2 Clases y Objetos

Un **clase** es un plantilla que define las propiedades y métodos para poder crear **objetos**. De este manera, un objeto es una instancia de una clase.

- Similar a otros lenguajes como Java o C#
- Representan algo real.
- Se declaran con la palabra reservada **class**
- Entre llaves se declaran los atributos y métodos, que pueden ser privados, públicos o protegidos. Por defecto son públicos
 - **private**: Sólo puede acceder la propia clase.
 - **protected**: Sólo puede acceder la propia clase o sus descendientes.
 - **public**: Puede acceder cualquier otra clase.



Buenas Practicas:

- Todas las clases empiezan por letra mayúscula.
- Siempre definiremos cada clase en un fichero que nombraremos *NombreClase.php*
- Los atributos siempre serán privados o protegidos (crearemos getters y setters para acceder a ellos)

Para poder crear variables u **objetos** de cualquier clase, necesitamos definir una función especial llamada **constructor**. Estas funciones pueden recibir una serie de parámetros, que normalmente son los valores que queremos asignarles a los distintos atributos.

- Para instanciar un objeto a partir de la clase, se utiliza *new*.
- Para acceder desde un objeto a sus atributos o métodos, utilizamos la operador **flecha (->)**
- El objeto **\$this**

- Cuando desde un objeto se invoca un método de la clase, a este se le pasa siempre una referencia al objeto que lo ha llamado. Esta referencia se almacena en la variable **\$this**
- Solo puede haber un **constructor** se llamará ***__construct***
 - Será invocada automáticamente al hacer *new*
 - Es ideal para inicializar los datos del objeto antes de usarlo.

Así, podemos definir una clase y un constructor que reciba tres parámetros (uno para el código, otro para el título y otro para la versión) y los asigne a los correspondientes atributos:

```
class Software
{
    private $codigo = 0;
    private $titulo = "";
    private $version = "1";

    public function __construct($c, $t, $v)
    {
        $this->codigo = $c;
        $this->titulo = $t;
        $this->version = $v;
    }

    ...
}
```

Con esto, ya podemos crear variables de este tipo, usando el operador **new** para crear cada objeto. Por ejemplo, así crearíamos dos variables de tipo `Software`, `$s1` y `$s2`, con diferentes datos cada una (desde fuera de la clase, si queremos):

```
$s1 = new Software(1, "LibreOffice", "6.0");
$s2 = new Software(2, "GIMP", "3.8");
```